

Reinforcement Learning for Latent-Space Thinking in LLMs

Enes Özeren¹, Matthias Assenmacher^{1,2}

¹LMU Munich, Department of Statistics, Germany

²Munich Center for Machine Learning (MCML), Germany

Correspondence: enozeren@gmail.com matthias@stat.uni-muenchen.de

Abstract

Chain-of-Thought (CoT) reasoning typically utilizes the discrete language space for thinking, which is inherently inefficient, as many generated tokens only enforce linguistic rules that are not required for reasoning. To bypass this, *latent-space thinking* allows models to think using the continuous embedding space. While existing methods for training those models show domain-specific gains, they fail to maintain performance in complex tasks, such as mathematical reasoning. We experimentally demonstrate that the Coconut approach, a form of supervised fine-tuning for latent-space thinking, is highly sensitive to design choices and exhibits several inherent limitations. To address these issues, we investigate reinforcement learning (RL) techniques — an under-explored direction in latent-space thinking — including GRPO and design a novel *Latent RL* method for directly optimizing the latent thinking steps. Our experimental results reveal that these RL-trained models still lag behind traditional language-space CoT models in the mathematical reasoning domain. We make our codebase publicly available.¹

1 Introduction

Language model (LM) training is typically divided into two phases: pre-training, where models acquire general language understanding and world knowledge, and post-training, which adapts them for downstream use cases such as dialogue, code completion, or translation (Grattafiori et al., 2024). Among the most widely used post-training methods is reinforcement learning (RL), which aligns LM behavior with human preferences (Ouyang et al., 2022; Touvron et al., 2023b). RL has also been applied to strengthen reasoning abilities in domains such as mathematics and programming (Shao et al., 2024). While effective, such methods incur substantial computational costs, as models trained in

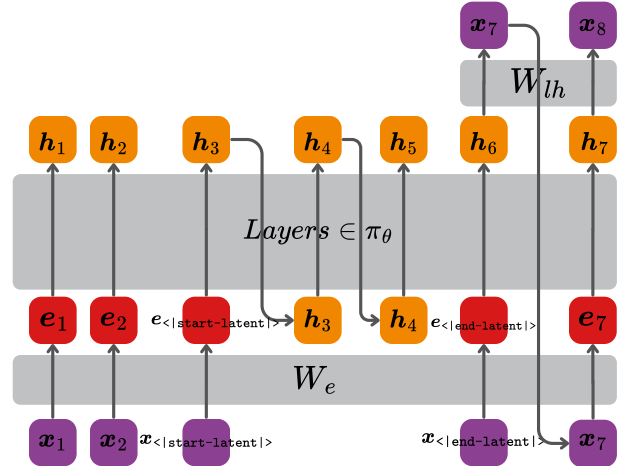


Figure 1: Illustration of latent-space thinking. Starting from the input (x_1, x_2), the $\langle |start-latent| \rangle$ is inserted, then the model switches to the latent mode and performs two latent steps ($C = 2$). Once the $\langle |end-latent| \rangle$ token is reached, the model switches back to language mode to answer in language-space (x_7, x_8).

this way tend to produce longer responses during inference (Guo et al., 2025; Zheng et al., 2025).

An alternative direction improves efficiency by shifting reasoning from the language space (discrete tokens) to the latent space (i.e., the continuous hidden states, see Fig. 1, Hao et al., 2025; Zhu et al., 2025b), inspired by neuroscience findings that language primarily serves communication rather than thought (Fedorenko et al., 2024). A notable example is the *Coconut* (Chain of Continuous Thought) method (Hao et al., 2025), which directly reuses final-layer embeddings as subsequent input, rather than decoding them into tokens. This allows the model to operate in its continuous latent space, avoiding the bottleneck imposed by discrete language tokens. However, training such models is non-trivial, as no ground-truth latent-space thinking² data exists. To address this, Coconut

¹github.com/enesozeren/latent-space-thinking-model

²With "Latent-Space Thinking" we refer to the same con-

follows the strategy of [Deng et al. \(2024\)](#), employing a supervised fine-tuning (SFT) approach where language-space reasoning steps are gradually replaced by latent-space thinking steps (referred to as *Coconut SFT* procedure). A crucial aspect of this procedure is that latent thinking steps do not receive direct supervision signals; instead, they are guided only through indirect signals during training. As a result, even though [Hao et al. \(2025\)](#) reported improvements in the logical reasoning domain, the performance drops in the mathematics domain when compared with the language-space thinking models.

In this work, we explore RL-based methods as a way to improve latent-space thinking models. Initial experiments applying group relative policy optimization (GRPO, [Shao et al., 2024](#)) show no measurable performance gains. As an alternative, we explore a novel RL method tailored to latent-space thinking to try to overcome the indirect learning signal limitation of the Coconut SFT method. However, this approach results in unstable training and fails to deliver performance improvements. Finally, we conduct an in-depth analysis of the Coconut SFT procedure, including ablation studies, which demonstrate that the procedure is sensitive to several factors, with performance being highly fragile. This analysis also evaluates several modifications, with the results indicating some promising adaptations as well as methodological weaknesses to be improved in future work.

The main contributions of this work are summarized as follows:

1. An empirical study of training latent-space thinking models with GRPO.
2. The introduction of a novel RL method tailored to latent-space thinking models.
3. An empirical in-depth analysis of the Coconut SFT procedure.

2 Related Work

Latent-Space Thinking In their recent survey paper, [Zhu et al. \(2025b\)](#) highlight the growing body of work on latent-space thinking. One of the earliest studies by [Goyal et al. \(2023\)](#) demonstrated that inserting dummy <pause> tokens before generating a response improves LM performance on

cept coined as "Latent Reasoning" by ([Hao et al., 2025](#)). In our opinion, this is more adequate, as only a few latent steps are probably not yet real reasoning.

some tasks. This finding suggests that using more compute, even without producing language tokens, is beneficial for enabling the model to create richer (internal) representations before generating the answer. Similarly, [Pfau et al. \(2024\)](#) showed that even adding meaningless tokens like ‘.’ (dot) can achieve performance comparable to Chain-of-Thought (CoT) reasoning in certain problems.

Instead of focusing on latent thinking, [Deng et al. \(2024\)](#) aimed to internalize the CoT process within LMs. They proposed an SFT procedure that gradually removes explicit CoT steps from the training data, allowing the model to internalize the CoT process. Their results showed that this approach outperforms models trained without CoT, but still falls short of the performance achieved by models trained with CoT steps. Building on this idea, [Hao et al. \(2025\)](#) introduced the Coconut method, which incorporates latent thinking steps while also gradually removing the CoT steps during SFT. The goal is to enable the model to reason in an unrestricted latent space, rather than in the discrete language space. They define *latent reasoning* as feeding the last hidden layer’s last embedding back to the model as input, rather than decoding it into a token and feeding the corresponding token embedding in the next autoregressive iteration, as typically done in standard text generation. Their experiments show that this approach provides slight improvements on certain logical reasoning benchmarks but performs worse than explicit CoT in language space on the mathematical benchmark GSM8K ([Cobbe et al., 2021](#)). [Zhu et al. \(2025a\)](#) build on Coconut by developing a theoretical framework to explain why latent-space thinking can enhance LM performance on certain tasks, with a particular focus on the graph reachability problem.

Reinforcement Learning One of the first applications of RL for LM post-training was proximal policy optimization (PPO, [Schulman et al., 2017](#)), which laid the groundwork for reinforcement learning from human feedback (RLHF, [Ziegler et al., 2019](#); [Ouyang et al., 2022](#)) for LMs. It aligns models with human preferences by increasing the likelihood of responses favored by annotators. Because collecting human feedback for every output is costly, a reward model is trained on limited annotations to assign a scalar reward to each complete response ([Ouyang et al., 2022](#)). However, this single score gives sparse feedback for long outputs. To refine this, a value model ([Mnih et al., 2016](#))

predicts token-level contributions to the reward, offering more fine-grained feedback and improving training efficiency.

Recently, exploration of RL methods to enhance the reasoning capabilities of LMs has gained momentum (Shao et al., 2024; Lambert et al., 2024; Guo et al., 2025; Zheng et al., 2025). One of the first effective open-source approaches, Shao et al. (2024), introduced the GRPO method, which uses rule-based rewards tailored to mathematics and coding tasks—based on answer correctness or unit-test success. They reported consistent improvements during training and found that GRPO encourages progressively longer reasoning chains. Despite extensive exploration of SFT strategies in latent-space thinking models, the application of RL in this context remains largely unexplored.

3 Methodology

3.1 Latent Space Thinking

A LM π_θ with parameters θ processes text by tokenizing it into t tokens, each represented as a one-hot vector over the vocabulary V , forming $X^t \in \mathbb{R}^{t \times |V|}$. Tokens are embedded via the embedding matrix $W_e \in \mathbb{R}^{|V| \times D}$:

$$E^t = X^t W_e \quad (1)$$

The embeddings are contextualized by the model layers:

$$H^t = \text{Layers}(E^t), \quad H^t \in \mathbb{R}^{t \times D} \quad (2)$$

To predict the next token, the final hidden state h_t is projected into the vocabulary space using the language head $W_{lh} \in \mathbb{R}^{|V| \times D}$:

$$y_t = W_{lh} h_t, \quad p_t = \text{softmax}(y_t) \quad (3)$$

A decoding strategy selects the next token, and the process repeats auto-regressively until the end-of-sequence token is generated.

Latent-space thinking, as proposed by Hao et al. (2025), modifies the first and final step of the autoregressive generation process as illustrated in Figure 1. As a starting point, three new tokens, $\langle \text{start-latent} \rangle$, $\langle \text{latent} \rangle$, $\langle \text{end-latent} \rangle$, are added to the vocabulary and their embeddings are initialized randomly in the embedding matrix W_e . The $\langle \text{latent} \rangle$ acts purely as a placeholder for latent steps, and its embedding is never used by the model. When an input text X^t is provided, the $\langle \text{start-latent} \rangle$

token is directly appended as the $(t + 1)$ -th token, forming X^{t+1} .

Instead of proceeding with token generation using the language head matrix W_{lh} and a decoding strategy, the latent-space thinking model performs a latent step by taking the last contextualized token embedding h_{t+1} and appending it directly to the embedded sequence E^{t+1} . This is done via row-wise concatenation operation, denoted as concat :

$$E^{t+2} = \text{concat}(E^{t+1}, h_{t+1}) \quad (4)$$

This process, referred to as latent-space thinking, is repeated until a predefined number C of latent steps have been performed. After that, the $\langle \text{end-latent} \rangle$ token is appended. Following this, the latent-space thinking model reverts to the standard token generation in language space. To implement this mechanism, we used the Huggingface transformers library (Wolf et al., 2019) with targeted modifications to the generate and forward methods.

3.2 Existing Training Methods

Supervised Fine-Tuning For the language-space thinking model, we apply standard SFT without any modifications. Given a question-response pair (X^{q_i}, X^{r_i}) , we do one forward pass and calculate the cross-entropy loss on each response token:

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^N \log(\pi_\theta(X^{r_i} | X^{q_i})) \quad (5)$$

For the latent-space thinking model, we follow the Coconut SFT procedure (Hao et al., 2025). Given (X^{q_i}, X^{r_i}) , the model first processes all tokens in the question X^{q_i} , then generates latent steps auto-regressively as described in Section 3.1. After the latent steps are generated, the embeddings of the question tokens E^{q_i} , the latent steps E^{l_i} , and the response tokens E^{r_i} are concatenated and passed through the model in a single forward pass. During training, a cross-entropy loss is computed over the response tokens only; question tokens and latent steps are masked and excluded from the loss. Thus, the objective minimized during the Coconut SFT procedure is:

$$\mathcal{L}_{\text{SFT}} = - \sum_{i=1}^N \log(\pi_\theta(X^{r_i} | X^{q_i}, E^{l_i})) \quad (6)$$

This method optimizes the latent steps indirectly through the language space tokens in the response part.

(Modified) GRPO Applying RL to latent-space thinking models represents an unexplored research direction. We therefore experiment with the GRPO algorithm to investigate its impact. The language-space thinking model is trained with the GRPO implementation from the Hugging Face TRL package (von Werra et al., 2020). In contrast, training the latent-space thinking model with GRPO requires a modified version of this module. We modify the TRL package to exclude latent-step embeddings from the GRPO loss calculations since the GRPO loss requires token probabilities.

Similar to the Coconut SFT procedure, this modified GRPO setup optimizes the latent steps indirectly through the probabilities of tokens in the language space, and thus only through the final answer portion. In GRPO training, two types of rewards are used: an accuracy and a format reward (see Appendix A).

3.3 Proposing Latent RL

Neither Coconut SFT nor GRPO provides explicit supervision for training latent steps, as it is not straightforward to determine what constitutes a “good” latent step, since these steps are represented by continuous vectors and no ground-truth latent vectors are available.

To address this challenge, we propose an alternative RL approach, referred to as *Latent RL*. In the Latent RL training setup, a value model $\rho_{\theta+\phi}$ is employed to directly optimize the latent steps. The value model $\rho_{\theta+\phi}$ shares some of its parameters θ with the underlying policy model π_{θ} and is extended with additional value head parameters ϕ on top.

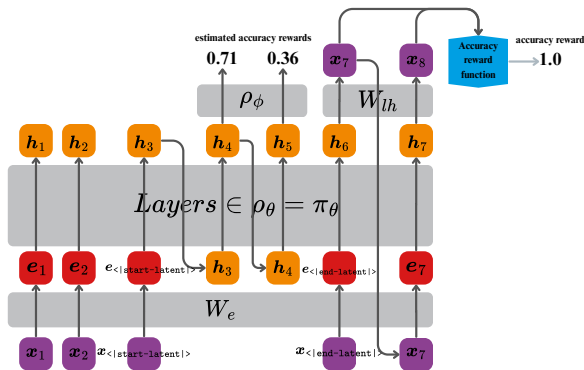


Figure 2: Illustration of Latent RL training. The value model head estimates the accuracy reward for each latent step.

The Latent RL training procedure is illustrated in Figure 2. In the first phase, we train $\rho_{\theta+\phi}$ by

freezing parameters θ . In the second phase, we train the policy model by utilizing the value model. To achieve this, we generate responses with latent steps for given questions and then maximize the value model predictions:

$$\arg \min_{\theta} L_{LAT}, \quad L_{LAT} = -\rho_{\theta+\phi}(\mathbf{h}) \quad (7)$$

Further details of the Latent RL procedure can be found in Appendix B.

3.4 Performance Measures

We primarily evaluate LMs for their mathematical capabilities and efficiency. Hence, the following two evaluation metrics are employed.

pass@k This metric estimates the expected probability that a model generates a correct answer to a question within its first k attempts (Chen et al., 2021; Guo et al., 2025; Dang and Ngo, 2025). It is defined as

$$\text{pass@}k = \mathbb{E}_{\text{problems}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right], \quad (8)$$

where n denotes the number of generated answers for a given question and c denotes the number of correct answers among them. We set $n = 4$ and report both pass@1 and pass@4.

Tokens The average number of tokens generated by the LM to produce an answer is a measure of output length and efficiency.

4 Experimental Setup

4.1 Models

The base LM used in this study is Qwen2.5 1.5B Base (Yang et al., 2024). This choice strikes a balance between computational feasibility and model capability in this study. For brevity, the model is referred to simply as Qwen Base. All methods and implementations proposed here are model-agnostic and can be applied to other decoder-only transformer LMs.

To analyze the effects of different training methodologies, we train and compare the models as outlined in Table 1.

4.2 Datasets

Since we employ two SFT and RL training methodologies, each with distinct requirements for the training data, we utilize different datasets for each of them (see Table 2).

Model Name	Training
Qwen Base	Base model
Qwen SFT	Qwen Base + SFT
Qwen SFT & GRPO	Qwen SFT + GRPO
LaST SFT	Qwen Base + Coconut SFT
LaST SFT & GRPO	LaST SFT + GRPO
LaST SFT & Latent RL	LaST SFT + Latent RL

Table 1: Models compared in this study. LaST is short for *Latent-Space Thinking Model*.

Dataset	Purpose	Usage Vol.
GSM8K-Aug-NL	Training (SFT)	300K
OpenR1-Math-220k	Training (RL)	10K
GSM8K	Testing	1.3K
MATH500	Testing	500

Table 2: The volumes of the datasets used in this study. K denotes thousands. For the training part, full datasets are not utilized for efficiency purposes. The Usage Vol. column reports the amount we used in our experiments.

For SFT, the primary requirement is that the dataset must contain question-response pairs. The Coconut SFT procedure, however, has the additional requirement that the responses X^{r_i} must be structured as a list of sequential steps. This means that, e.g., the response should contain a sequence of sentences or mathematical formulations, each corresponding to an individual, *separate* reasoning step. To satisfy these requirements, the GSM8K-Aug-NL dataset (Deng et al., 2023) is selected for SFT.

For pure RL training methods, a dataset containing question-answer pairs (X^{q_i}, X^{a_i}) is sufficient; the full response³ is not necessary. We use OpenR1-Math-220k (Hugging Face, 2025).

4.3 Training Details

The models in Table 1 are trained using the datasets in Table 2. The details of the training are given in this section. All the training hyperparameters are provided in Appendix C.

Qwen SFT. This model is obtained by SFT training of the Qwen Base model on 300K question-response pairs from GSM8K-Aug-NL.

Qwen SFT & GRPO. This model is trained with the GRPO algorithm using rule-based rewards, starting from the Qwen SFT model. The training

³We strictly differentiate *answers* (just the final result) from *responses* (including the whole reasoning trace) to emphasize the different requirements.

is performed on 10K samples from OpenR1-Math-220k.

LaST SFT. This model is trained using the Coconut SFT procedure proposed by Hao et al. (2025), starting from the Qwen SFT model and using the same 300K question-response pairs from GSM8K-Aug-NL as in the Qwen SFT training process. This procedure follows a multi-stage approach: in each stage, one language step (typically one sentence) from the ground-truth response is removed and replaced with two latent steps. In the final stage, all language steps are removed, and the model relies solely on latent steps to predict the final answer. Consistent with Hao et al. (2025), a three-stage approach was adopted, allowing the model to use up to six latent steps.

A key difference to Hao et al. (2025) lies in the choice of training data: instead of the GSM8K-Aug dataset, which contains only mathematical calculation steps, the GSM8K-Aug-NL dataset was used, where the steps consist of both language tokens and mathematical calculations. This choice was made to align with the main motivation for introducing latent steps - namely, to enable the model to internalize not only mathematical calculations but also language.

LaST SFT & GRPO. This model is trained on the same 10K question-answer pairs from the OpenR1-Math-220k dataset as used for the Qwen SFT & GRPO model, but starting from the LaST SFT model.

LaST SFT & Latent RL. The Latent RL method consists of two stages. First, the value model is trained to estimate accuracy rewards for latent steps. For this purpose, 20K question-answer pairs from the OpenR1-Math-220k are selected, and responses are generated with the LaST SFT model. Accuracy rewards for the generated responses are computed using the reward function, and the value model is trained to predict these rewards from the latent steps. The value head itself consists of a linear layer that maps each latent step embedding to a scalar score representing the estimated reward. In the second stage, training is performed using 10K question-answer pairs from the OpenR1-Math-220k dataset, distinct from those used for value model training. These pairs are the same pairs as used in the LaST SFT & GRPO model training to be consistent.

Model	Shot	GSM8K			MATH500		
		pass@1	pass@4	# Tokens	pass@1	pass@4	# Tokens
Qwen Base	Zero	24.8	49.7	156	12.1	27.4	310
Qwen Base	One	57.0	73.4	575	25.4	40.2	589
Qwen SFT	Zero	62.6	76.0	105	22.0	32.0	290
Qwen SFT & GRPO	Zero	72.6	82.5	216	38.7	50.0	366
LaST SFT	Zero	22.6	29.9	19	8.5	12.4	19
LaST SFT & GRPO	Zero	21.8	24.1	19	9.2	14.0	18
LaST SFT & Latent RL	Zero	22.7	30.1	19	8.4	13.2	19

Table 3: Evaluation results on GSM8K and MATH500 benchmarks.

5 Results

This chapter summarizes the main conclusions drawn from the experiments (cf. Table 3) and an ablation study for Coconut SFT.

5.1 Main Results

Evaluating Qwen Base is challenging. In the zero-shot setting, the Qwen Base model often fails to follow the instructed answer format, making it difficult to evaluate performance based on boxed answers in the answer block (see Appendix D for more details). To address this, the model was also tested in a one-shot setting with a single example showing the expected format. However, prior work shows that few-shot prompting, including one-shot, can boost performance beyond format compliance (Brown et al., 2020; Wei et al., 2022), so the model’s true zero-shot ability likely lies between its zero- and one-shot results presented here. Notably, in one-shot evaluation, the model sometimes hallucinated additional questions and answers, resulting in notably longer responses.

SFT on GSM8K-Aug-NL improves the performance on GSM8K. The Qwen SFT model outperforms the Qwen Base model under both zero-shot and one-shot prompting on the GSM8K benchmark. This improvement can be attributed to the use of the GSM8K-Aug-NL dataset during SFT, which contains questions similar to those in GSM8K and thereby enhances domain-specific performance. On the MATH500 benchmark, however, the Qwen SFT model surpasses the Qwen Base model under zero-shot prompting but performs worse than under one-shot prompting. This indicates that the fine-tuning provides less benefit for MATH500, suggesting more limited transfer

compared to the gains observed on GSM8K.

GRPO on OpenR1-Math-220k improves the performance and increases the response length. The Qwen SFT & GRPO model, obtained through training with the GRPO algorithm with rule-based rewards, achieves higher performance on both benchmarks compared to the Qwen SFT model. In addition, the average response length increases after GRPO training, consistent with previous studies (Guo et al., 2025; Dang and Ngo, 2025).

Coconut SFT harms model performance. Similar to the findings in Hao et al. (2025), the LaST SFT model trained with the Coconut SFT procedure performs worse than the Qwen SFT model on mathematical benchmarks. And since the model only produced six latent steps prior to the answer block, the number of tokens generated is significantly lower than other language-space thinking models.

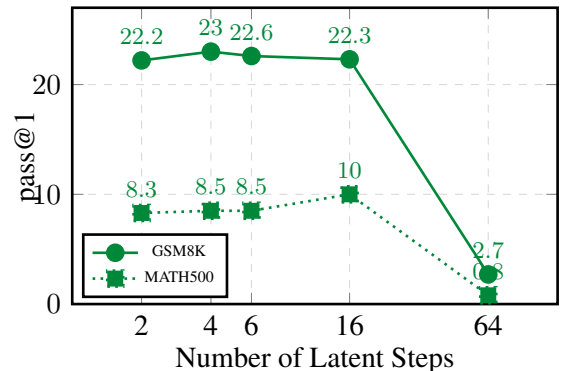


Figure 3: Effect of varying the number of latent steps on the pass@1 score with the LaST SFT model. The x-axis has the log scale.

To examine the influence of the number of latent steps in the LaST SFT model, evaluations were con-

ducted using configurations of 2, 4, 6, 16, and 64 latent steps, despite the model having been trained with a maximum of 6 latent steps (Figure 3). The results show that increasing the number of latent steps does not improve performance. Although the model stays within distribution up to 16 latent steps, its performance declines sharply at 64 latent steps.

GRPO training does not improve the performance of LaST SFT. Using the GRPO algorithm, which ignores the latent steps, does not yield performance gains for the LaST model, in contrast to results for the Qwen SFT & RL model. Two main factors may explain this outcome:

First, the GRPO algorithm is designed for language-space reasoning models, where it increases the probability of generating intermediate language tokens that eventually lead to correct answers. In the LaST SFT model, however, no such intermediate language tokens are produced before the final answer, which prevents the learning signal from being effective. To address this limitation, we introduced Latent RL in Section 3.3. Second, GRPO typically enhances performance by encouraging models to generate longer responses before the final answer, thereby increasing inference-time computation. In the LaST SFT model, the number of latent steps is fixed. However, as shown in Figure 3, simply increasing the number of latent steps does not enhance performance. This indicates that the effect of this factor is limited.

Latent RL training is unstable. Even though we address the limitations of other training techniques for latent-space thinking model training, we observe that Latent RL training shows unstable optimization dynamics (Appendix E). This is also reflected in the benchmark evaluations, where performance does not increase despite the Latent RL training. These observations suggest that the unstable Latent RL struggles to improve latent-space thinking. Exemplary model outputs for a question are presented in Appendix F.

5.2 Coconut SFT Ablations

We conduct ablation studies for the Coconut SFT procedure, as the baseline latent-space thinking model is trained with it.

Necessity of the Coconut SFT Procedure. When latent-space thinking is introduced to the Qwen Base model without any additional training, the model produces out-of-distribution and nonsen-

sical outputs (Figure 4). This occurs because the model is fed its own latent steps as input—data it has never encountered during pre-training. Therefore, an SFT procedure is essential before applying any RL training, as RL relies on having at least some correct responses. To this end, we employ the Coconut SFT method (Hao et al., 2025).

Qwen Base with Latent-Steps

```
<|start-latent|><|latent|><|end-latent|>
[\\{\\}\\$);,;\\{degrees\\} ( ( ); shi zi equ zi
. hua\\}\\<|end-latent|><|end-latent|>
.all\\_()\\{\\}
```

Figure 4: Qwen Base response to the question in Figure 10 after introducing latent-steps.

Language Removal Approach. Instead of a step-wise removal of pre-defined reasoning steps (as in Coconut SFT), we investigate a token-wise removal approach, where a fixed number of tokens is removed. This relaxes dataset constraints, enabling the use of any question-response dataset for latent-space thinking model training beyond GSM8K-Aug-NL. For this investigation, all reasoning steps in GSM8K-Aug-NL answers are concatenated. During LaST SFT training, we remove 12 tokens per latent step—half the average step length—to match the step-wise setup.

Removal Technique	GSM8K pass@1	MATH500 pass@1
Step-wise	17.9	11.3
Token-wise	7.5	3.4

Table 4: Investigation of different language token removal techniques in Coconut SFT.

Step-wise removal performs better (Table 4), presumably because token-wise removal can truncate reasoning steps mid-thought, making it more difficult for the model to internalize the reasoning process.

Number of Epochs per Stage. The Coconut SFT method uses 3 epochs per stage, with each stage involving one language step removal and two latent space insertions. We explore different numbers of epochs per stage:

Training the LaST model with one epoch per stage performs better than with two epochs (Table 5). In comparison, the Coconut method (Hao et al., 2025) reported using 3 epochs per stage. Fur-

Epochs/Stage	GSM8K pass@1	MATH500 pass@1
1	17.9	11.3
2	17.0	10.4

Table 5: Ablation study on num. of epochs/stage.

ther experiments were omitted due to the slow training of Coconut SFT.

Final Stage Training Epochs Effect. We also explore the impact of each epoch on the final stage of Coconut SFT procedure, showing that this last stage enhances the performance of the LaST SFT model on the GSM8K benchmark (Figure 5). However, the observed performance drop on the MATH500 benchmark indicates that extended training on the GSM8K-Aug-NL dataset reduces the generalization.

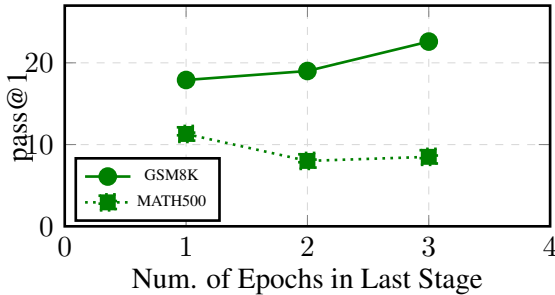


Figure 5: The effect of number of epochs in the last stage of the Coconut SFT procedure.

6 Conclusion

In this work, we explored different training strategies for latent-space thinking models, starting from the Coconut SFT procedure (Hao et al., 2025) and extending it to RL-based methods. While Coconut SFT provides a structured way to train latent-space thinking models, it consistently underperforms compared to language-space models on mathematical benchmarks. We showed, that its success depends heavily on design choices, such as the number of epochs in the final stage and token removal strategies. The method’s main limitation lies in the absence of direct learning signals to latent steps and its reliance on auto-regressive processing, which limits scalability.

RL approaches were then evaluated as potential improvements. Extending GRPO to Coconut-trained latent-space thinking models brought no gains, since GRPO also lacks direct latent-step su-

pervision. To overcome this, we introduced the Latent RL method, which provides explicit learning signals to latent steps through a value model. However, training was unstable and did not improve benchmark performance.

Future work should aim to enhance SFT training efficiency, stabilize Latent RL, and develop mechanisms for adaptive control over latent steps. Overall, the findings suggest that current latent-space thinking training methods still lag behind language-space models, primarily due to their instability or the absence of supervision for latent-space thinking.

Limitations

This study examines training techniques for latent-space thinking models in the mathematical domain. While this focus allows for precise evaluation, it limits generalizability to other tasks such as general knowledge, logical reasoning, or programming. Prior work (Hao et al., 2025; Zhu et al., 2025a) suggests that latent-space thinking may perform differently in those domains, highlighting a key direction for future research.

Another limitation is the use of a single base model, which ensures consistency but narrows the scope of conclusions. Models with different sizes, architectures, or pretraining strategies may respond differently to these training methods, and exploring such variations could offer deeper insight.

A further challenge is efficiency: unlike standard LMs, latent-space thinking models require auto-regressive generation during both training and inference, increasing computational cost and limiting scalability. This makes large-scale training impractical with current setups.

Finally, latent-space thinking reduces model interpretability, making it harder to analyze internal processes or ensure transparency in deployment. Future work should therefore focus on improving interpretability—such as by studying attention or representation changes across latent steps—to better understand and control latent-space thinking behavior.

Acknowledgments

We gratefully acknowledge the Leibniz Supercomputing Centre (LRZ) of the Bavarian Academy of Sciences and Humanities and the Munich Center for Machine Learning (MCML) for generously providing computational resources. Matthias

Aßenmacher received funding from the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under the National Research Data Infrastructure – NFDI 27/1 - 460037581 - BERD@NFDI.

References

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, and 1 others. 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, and 1 others. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Quy-Anh Dang and Chris Ngo. 2025. Reinforcement learning for reasoning in small llms: What works and what doesn’t. *arXiv preprint arXiv:2503.16219*.
- Yuntian Deng, Yejin Choi, and Stuart Shieber. 2024. From explicit cot to implicit cot: Learning to internalize cot step by step. *arXiv preprint arXiv:2405.14838*.
- Yuntian Deng, Kiran Prasad, Roland Fernandez, Paul Smolensky, Vishrav Chaudhary, and Stuart Shieber. 2023. Implicit chain of thought reasoning via knowledge distillation. *arXiv preprint arXiv:2311.01460*.
- Evelina Fedorenko, Steven T Piantadosi, and Edward AF Gibson. 2024. Language is primarily a tool for communication rather than thought. *Nature*, 630(8017):575–586.
- Sachin Goyal, Ziwei Ji, Ankit Singh Rawat, Aditya Krishna Menon, Sanjiv Kumar, and Vaishnavh Nagarajan. 2023. Think before you speak: Training language models with pause tokens. *arXiv preprint arXiv:2310.02226*.
- Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and 1 others. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason E Weston, and Yuandong Tian. 2025. Training large language models to reason in a continuous latent space. In *Second Conference on Language Modeling*.
- Hugging Face. 2025. Open r1: A fully open reproduction of deepseek-r1.
- Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, and 1 others. 2024. Tulu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*.
- Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*.
- Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. 2016. Asynchronous methods for deep reinforcement learning. In *International conference on machine learning*, pages 1928–1937. PmlR.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, and 1 others. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744.
- Jacob Pfau, William Merrill, and Samuel R Bowman. 2024. Let’s think dot by dot: Hidden computation in transformer language models. *arXiv preprint arXiv:2404.15758*.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Y Wu, and 1 others. 2024. Deepseek-math: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, and 1 others. 2023b. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, Nathan Lambert, Shengyi Huang, Kashif Rasul, and Quentin Galouédec. 2020. Trl: Transformer reinforcement learning. <https://github.com/huggingface/trl>.

- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, and 1 others. 2022. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, and 1 others. 2019. Huggingface’s transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, and 1 others. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, and 1 others. 2025. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*.
- Hanlin Zhu, Shibo Hao, Zhiting Hu, Jiantao Jiao, Stuart Russell, and Yuandong Tian. 2025a. Reasoning by superposition: A theoretical perspective on chain of continuous thought. *arXiv preprint arXiv:2505.12514*.
- Rui-Jie Zhu, Tianhao Peng, Tianhao Cheng, Xingwei Qu, Jinfa Huang, Dawei Zhu, Hao Wang, Kaiwen Xue, Xuanliang Zhang, Yong Shan, and 1 others. 2025b. A survey on latent reasoning. *arXiv preprint arXiv:2507.06203*.
- Daniel M Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. 2019. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*.

A Reward Functions

For the GRPO method, accuracy and format rewards are used; while the accuracy reward is the same, the format reward differs as indicated below. The Latent RL method uses the same reward functions as GRPO for the latent-space thinking model.

Accuracy Reward

Reward	Condition
$r = 1$	The correct answer appears in <code>\boxed{...}</code> inside <code><answer>...</answer></code> tags.
$r = 0$	Otherwise.

The correctness is verified by exact string match or mathematical equivalence similar to [Hugging Face \(2025\)](#).

Format Reward — Language-Space Thinking Models

Reward	Format
$r = 1$	<code><think>...</think></code> <code><answer> \boxed{...} </answer></code>
$r = 0.5$	Same format but the <code>\boxed{...}</code> is missing.
$r = 0$	Otherwise.

Format Reward — Latent-Space Thinking Models

Reward	Format
$r = 1$	<code>< start-latent >...< end-latent ></code> <code><answer> \boxed{...} </answer></code>
$r = 0.5$	Same format but the <code>\boxed{...}</code> part is missing.
$r = -1$	Any of <code>< start-latent ></code> , <code>< end-latent ></code> , <code><think></code> appear after the first latent block.
$r = 0$	Otherwise.

B Latent RL Methodology

Latent RL method has two stages:

1. Training a value model $\rho_{\theta+\phi}$ by freezing parameters θ .
2. Training the policy model π_{θ} by leveraging the predictions of the value model for latent steps.

In the first stage, data is collected to train $\rho_{\theta+\phi}$. A policy model with latent space thinking π_{ϕ} is used to generate responses to questions. For each response, an accuracy reward is computed using

the reward function described in Section A. From these responses, the final-layer embeddings $\mathbf{h}$ are extracted and paired with the corresponding accuracy reward r , forming training pairs $(\mathbf{h}_i, r_i)$.

The value model $\rho_{\theta+\phi}$ maps the embedding $\mathbf{h}$ to an estimated reward $\rho_{\theta+\phi}(\mathbf{h})$. To train it, only the value head parameters ϕ are optimized by minimizing the binary cross-entropy (BCE) loss L_{BCE} over embedding–reward pairs $(\mathbf{h}_i, r_i)$. Crucially, the loss is computed exclusively on embeddings corresponding to latent steps, while embeddings from other parts of the sequence—such as the question tokens or the final language answer—are ignored. This ensures that the value model learns to evaluate the quality of latent-space thinking steps.

$$\arg \min_{\phi} L_{\text{BCE}}(\rho_{\theta+\phi}) \quad (9)$$

$$\begin{aligned} L_{\text{BCE}}(\rho_{\theta+\phi}) = & r_i \log(\rho_{\theta+\phi}(\mathbf{h}_i)) \\ & + (1 - r_i) \log(1 - \rho_{\theta+\phi}(\mathbf{h}_i)) \end{aligned} \quad (10)$$

In the second stage, the policy model π_{θ} is provided with a question and generates a sequence consisting of latent steps followed by a final language answer. This complete sequence—comprising the given question, the latent steps, and the generated answer—is then passed to the value model $\rho_{\theta+\phi}$, ensuring that it receives the same input as the policy model. Although the value model outputs estimated rewards for all positions in the sequence, the training objective is applied only to the latent step positions. The goal is to maximize the estimated reward of the latent steps, which is equivalent to minimizing L_{LAT} in Equation (7).

When optimizing Equation (7), the value head parameters ϕ are frozen. This is necessary because if ϕ were also updated, the value head could trivially learn to always predict high rewards, thereby collapsing the training signal. Instead, only the shared parameters θ are updated, which means that optimizing the value model $\rho_{\theta+\phi}$ also updates the policy model π_{θ} , since the two models share θ .

After this update step, the value head parameters ϕ are refined separately using the BCE objective in Equation (9), with the true accuracy reward from that step as the target. This ensures that the value head remains aligned with the updated policy and continues to provide reliable reward estimates.

C Training Hyperparameters

The training set ups are presented in Table 6 and 7. All training runs use AdamW optimizer (Loshchilov and Hutter, 2017).

D Instruction Following

The ratio of generated responses that are fitting to the instructed format is given in Figure 6.

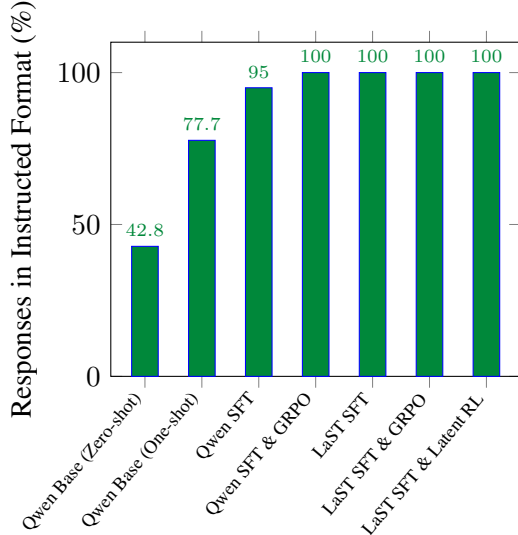


Figure 6: Ratio of answers following the instructed format during evaluation of models in GSM8K and MATH500 benchmarks. All models evaluated in zero-shot setting unless otherwise indicated.

E Latent RL Experiments

The training curves in Latent RL training procedure for the LaST SFT & Latent RL model in in Table 3 are presented in this section.

As described in Section 3.3, in the first phase we train a value model (Figure 7). Although the model achieves a high ROC-AUC score, its F1 score remains comparatively lower. This discrepancy arises because ROC-AUC measures the overall ranking ability of the model across thresholds, whereas the F1 score reflects the balance of precision and recall at a fixed threshold. Since the second phase of Latent RL training relies on the predicted probabilities rather than thresholded labels, ROC-AUC is a more informative indicator of value head performance in this setting.

After training the value model, in the second stage both the policy model and the value models are trained sequentially. However, we see that the

both policy loss and value head loss are fluctuating, indicating an unstable training (Figure 8).

To stabilize training, we freeze the value head during the second phase of Latent RL. This prevents it from becoming a moving target and leads to smoother optimization for the policy model, as shown in Figure 9. Although the policy loss decreases steadily, overall accuracy does not improve.

This stagnation arises from two causes: misalignment between the frozen value head and the evolving policy, and reward hacking. As the policy changes, the static value head becomes less accurate, while the policy exploits its fixed reward signal instead of improving reasoning or task accuracy. Thus, smoother training does not yield better benchmark performance.

F Example Responses from MATH500 Benchmark

Example answers from the models reported in Table 3 are shown in Figure 11 to Figure 16 for a representative question from the MATH500 benchmark, illustrating their behavior.

For this MATH500 example, the language-space models show variation in their responses, whereas all latent-space thinking models produce identical final answer. Since the latent steps are continuous, changes in their behavior cannot easily be inferred without additional interpretability studies.

Hyperparameter	Qwen SFT	LaST SFT
Dataset	GSM8K-Aug-NL	GSM8K-Aug-NL
Num Examples	300K	300K
Base Model	Qwen Base	Qwen Base
Hardware	4 × Nvidia A100-80GB	8 × Nvidia A100-40GB
Training Time	~ 1 hour	~ 8 hours
Num Epochs	1	5
Per Device Batch Size	4	1
Gradient Accum. Steps	32	128
Effective Batch Size	512	512
Learning Rate	5e-5	5e-5
LR Scheduler	StepLR	StepLR
LR Scheduler Args	$\gamma = 0.8$, step size=50	$\gamma = 0.8$, step size=50
Warmup Steps	—	—
Weight Decay	0.01	0.01

Table 6: Training hyperparameters for Qwen SFT and LaST SFT models. Note that the optimizer and scheduler reset after each stage in LaST SFT training.

Hyperparameter	Qwen SFT & GRPO	LaST SFT & GRPO	LaST SFT & Latent RL
Dataset	OpenR1-Math-220k	OpenR1-Math-220k	OpenR1-Math-220k
Num Examples	10K	10K	10K
Base Model	Qwen SFT	LaST SFT	LaST SFT
Hardware	4 × Nvidia A100-80GB	4 × Nvidia A100-80GB	1 × Nvidia A100-40GB
Training Time	~ 4 hours	~ 1 hour	~ 1.5 hours
Num Epochs	1	1	1
Batch Size / Device	2	4	2
Grad. Accum. Steps	32	16	128
Effective Batch Size	256	256	256
Learning Rate	5e-6	5e-6	5e-6
LR Scheduler	Cosine With Min LR	Cosine With Min LR	Cosine With Min LR
LR Scheduler Args	min lr rate = 0.1	min lr rate = 0.1	min lr rate = 0.1
Warmup Steps	10	10	10
Weight Decay	0.01	0.01	0.01
Max Completion Len.	1024	1024	1024
Temperature	0.7	0.7	0
Num Generations	8	8	1
KL Div. Coeff. β	0.0	0.64	—

Table 7: Training hyperparameters for Qwen SFT & GRPO and LaST SFT & GRPO models. Note that the effective batch size contains 256 responses to $256/8 = 32$ different questions and 8 responses for each questions in GRPO training.

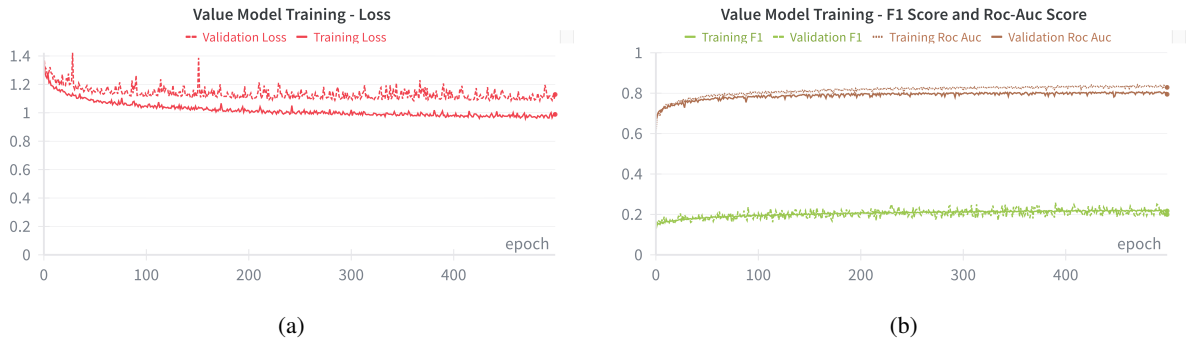


Figure 7: Training curves for the value model training in the first stage of Latent RL. Note that only the linear value head is trained. (a) BCE loss. (b) F1 and Roc Auc scores.

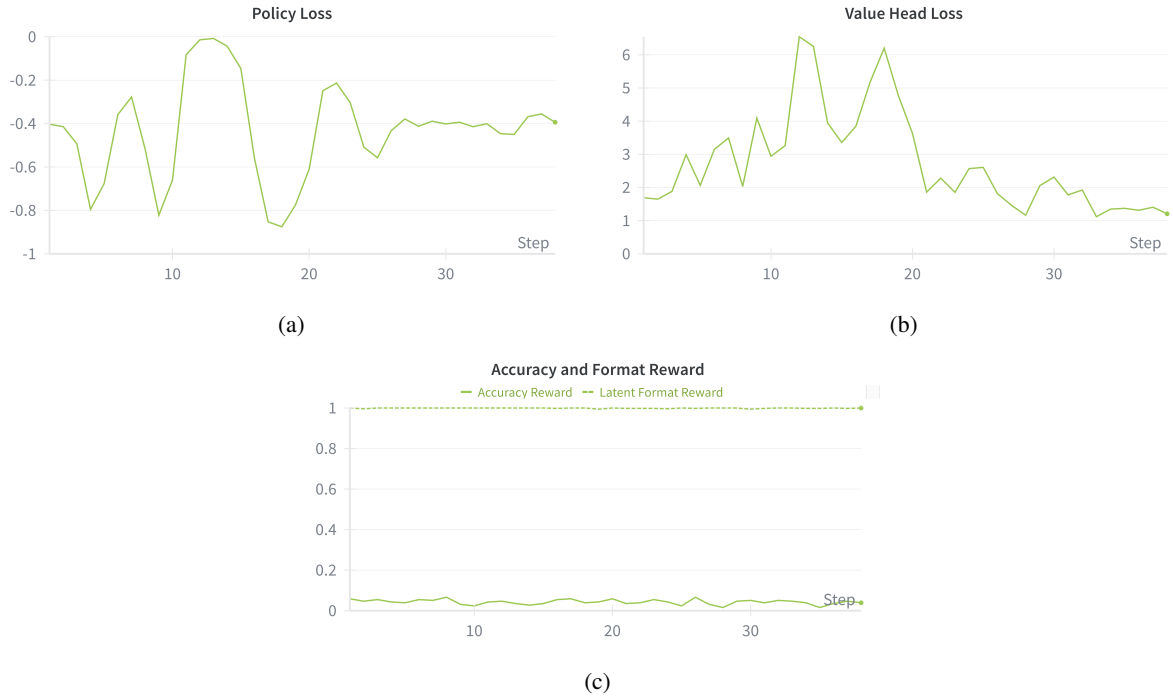


Figure 8: Training details of Latent RL: (a) policy loss, (b) value head BCE loss, (c) accuracy and latent format rewards.

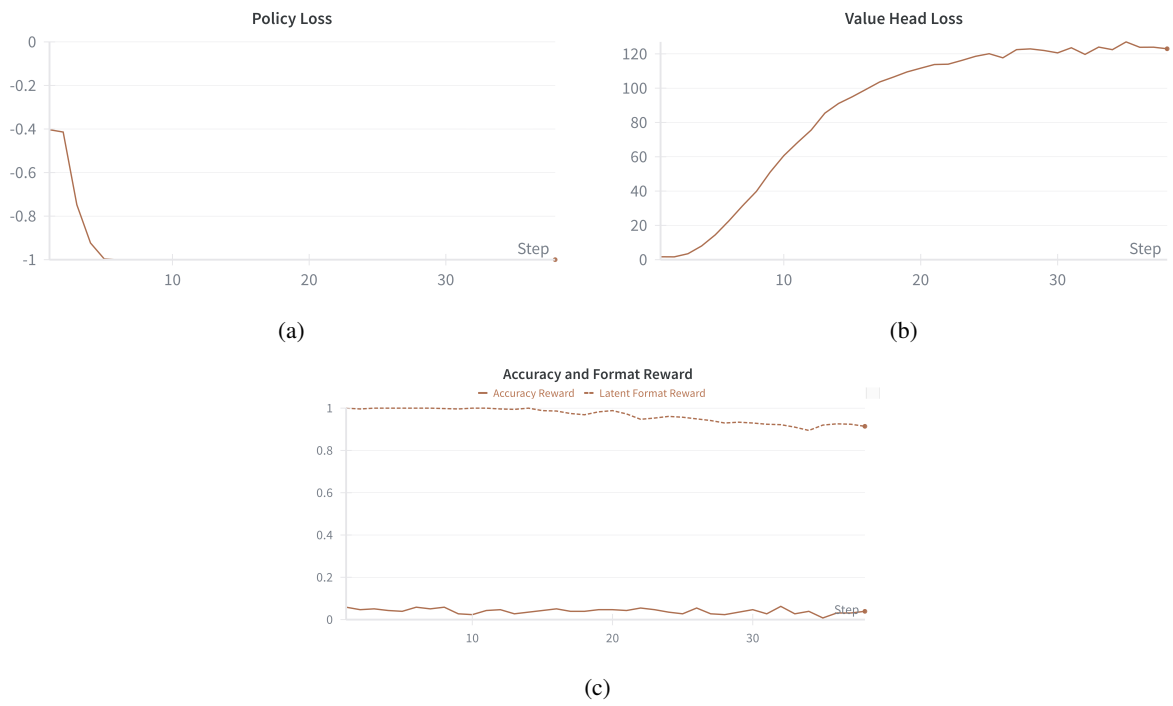


Figure 9: Training dynamics of Latent RL with a frozen value model. (a) Policy loss shows smooth convergence. (b) Value model loss, monitored but not optimized. (c) Accuracy and latent format rewards.

Question

How many positive whole-number divisors does 196 have?

Figure 10: Example question from MATH500 benchmark. The question, inserted into the system prompt which is omitted here for brevity. The correct answer is 9.

Qwen Base Response

```
<think>196 is a perfect square, so it has an even number of divisors. The prime factorization of 196 is  $2^2 * 7^2$ . The number of divisors is  $(2+1)(2+1) = 9$ . So, 196 has 9 positive whole-number divisors.</think><answer>\boxed{9}</answer>
```

Figure 11: Example Qwen Base (zero-shot) response to the question in Figure 10.

Qwen SFT Response

```
<think>First prime factorize  $196=2^2*7^2$ . The prime factorization of any divisor of 196 can only include the primes 2 and 7. We are free to choose either 0, 1, 2, or 3 as the exponent of 2 (4 options). For the exponent of 7, we can choose 0 or 1 (2 options). In total, there are  $4 \times 2=8$  possibilities.</think><answer>\boxed{8}</answer>
```

Figure 12: Example Qwen SFT response to the question in Figure 10.

Qwen SFT & GRPO Response

```
<think>The problem requires us to find the number of positive whole-number divisors of 196. First, we need to determine the prime factorization of 196. We start by dividing 196 by the smallest prime number, which is 2:
```

```
\[196 \div 2 = 98\]
```

Next, we divide 98 by 2:

```
\[98 \div 2 = 49\]
```

Now, 49 is not divisible by 2, so we check the next smallest prime number, which is 7:

```
\[49 \div 7 = 7\]
```

And again, 7 is a prime number. Therefore, the prime factorization of 196 is:

```
\[196 = 2^2 \times 7^2\]
```

To find the number of positive whole-number divisors, we use the formula that if a number has a prime factorization of the form $(p_1^{e_1} \times p_2^{e_2} \times \dots \times p_k^{e_k})$, then the number of divisors is given by $((e_1 + 1)(e_2 + 1) \times \dots (e_k + 1))$. For the factorization $(2^2 \times 7^2)$, the exponents are 2 and 2. Adding 1 to each of these exponents gives 3 and 3. Therefore, the number of divisors is:

```
\[(2+1)(2+1) = 3 \times 3 = 9\]
```

```
Thus, the number of positive whole-number divisors of 196 is \boxed{9}.
```

Figure 13: Example Qwen SFT & GRPO response to the question in Figure 10.

LaST SFT Response

```
<|start-latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|end-latent|><answer>\boxed{14}</answer>
```

Figure 14: Example LaST SFT response to the question in Figure 10.

LaST SFT & GRPO Response

```
<|start-latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|end-latent|><answer>\boxed{14}</answer>
```

Figure 15: Example LaST SFT & GRPO response to the question in Figure 10.

LaST SFT & Latent RL Response

```
<|start-latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|latent|><|end-latent|><answer>\boxed{14}</answer>
```

Figure 16: Example LaST SFT & Latent RL response to the question in Figure 10.